

DS5899: Special Topics in Data Science

Leveraging NLP in Asset Management 2

Week 6: Training

Instructor: Che Guan, Ph.D.

Yann LeCun's Cake Analogy on Self-supervised Learning, Supervised Learning, and Reinforcement Learning

Y. LeCun

How Much Information is the Machine Given during Learning?

▶ “Pure” Reinforcement Learning (cherry)

- ▶ The machine predicts a scalar reward given once in a while.

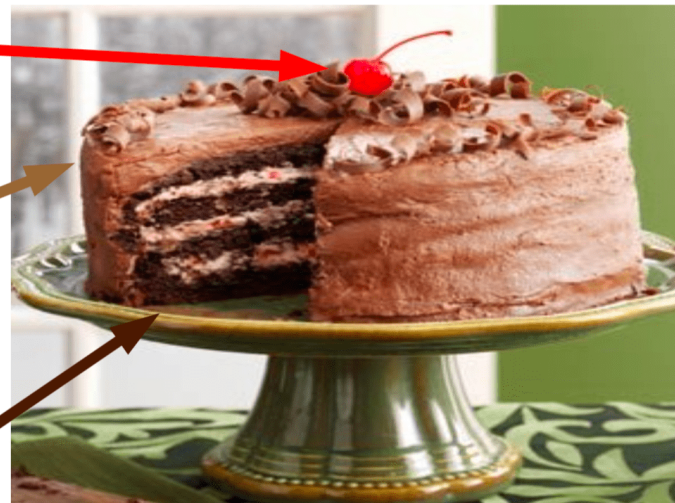
▶ A few bits for some samples

▶ Supervised Learning (icing)

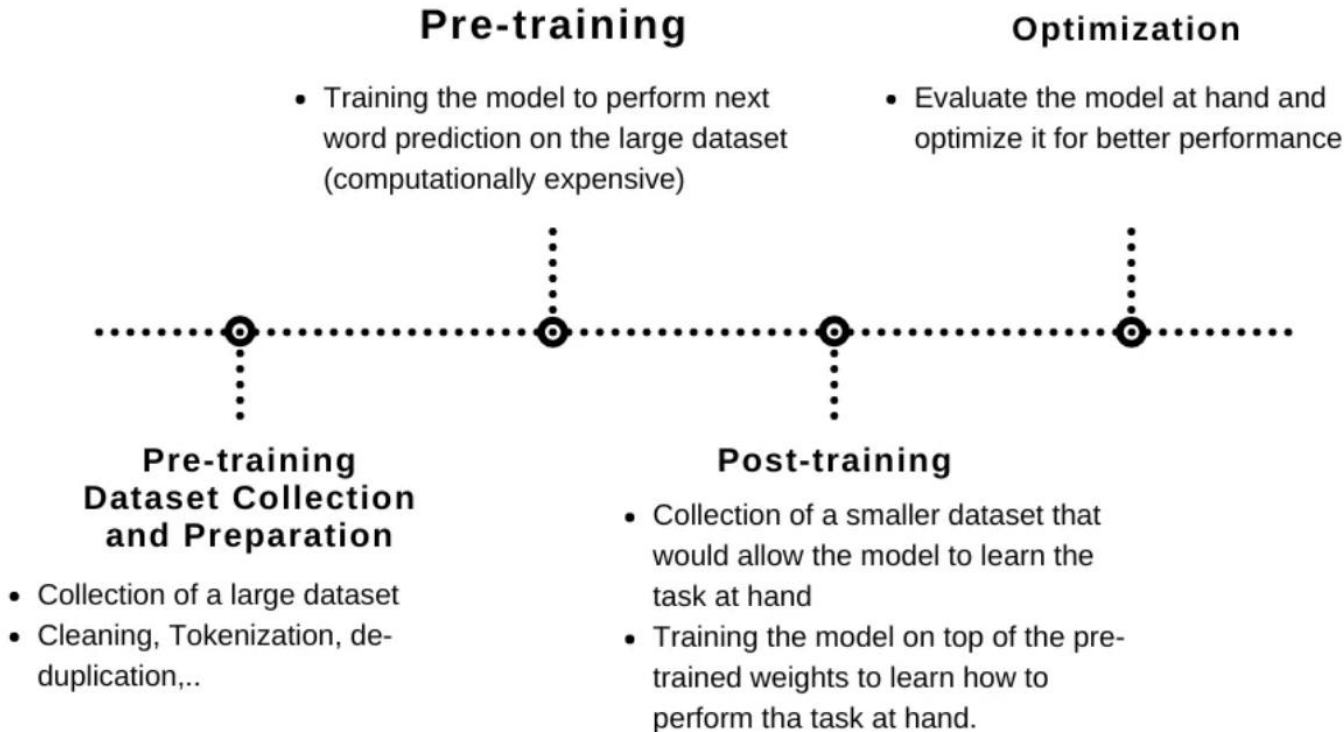
- ▶ The machine predicts a category or a few numbers for each input
- ▶ Predicting human-supplied data
- ▶ 10→10,000 bits per sample

▶ Self-Supervised Learning (cake génoise)

- ▶ The machine predicts any part of its input for any observed part.
- ▶ Predicts future frames in videos
- ▶ Millions of bits per sample



LLM Training Process



LLM Training Paradigms

- Modern LLMs, like OpenAI's GPT series, are developed through a two-phase process:
 - Pre-training for foundational knowledge
 - Post-training for alignment with human expectations.

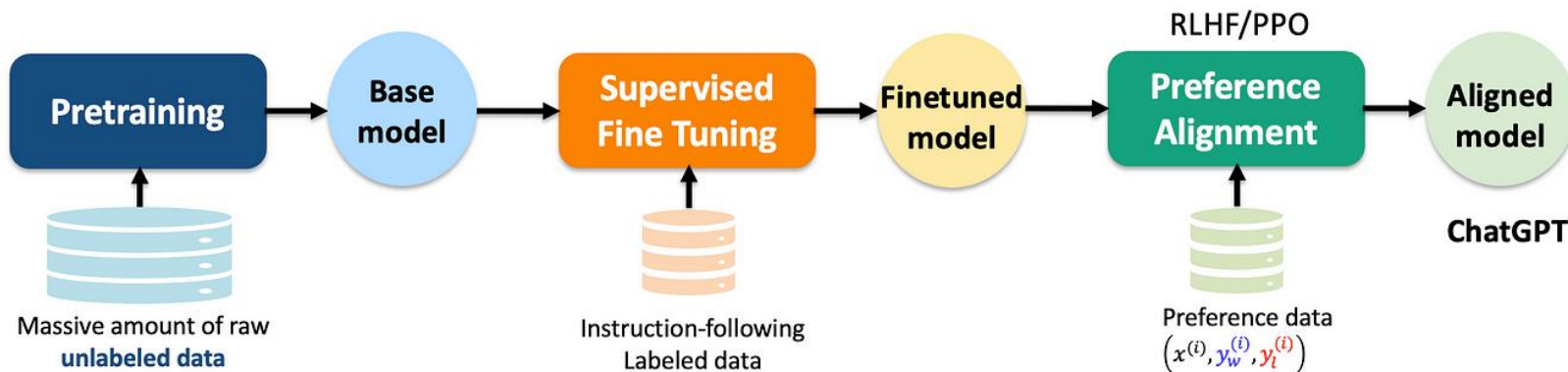
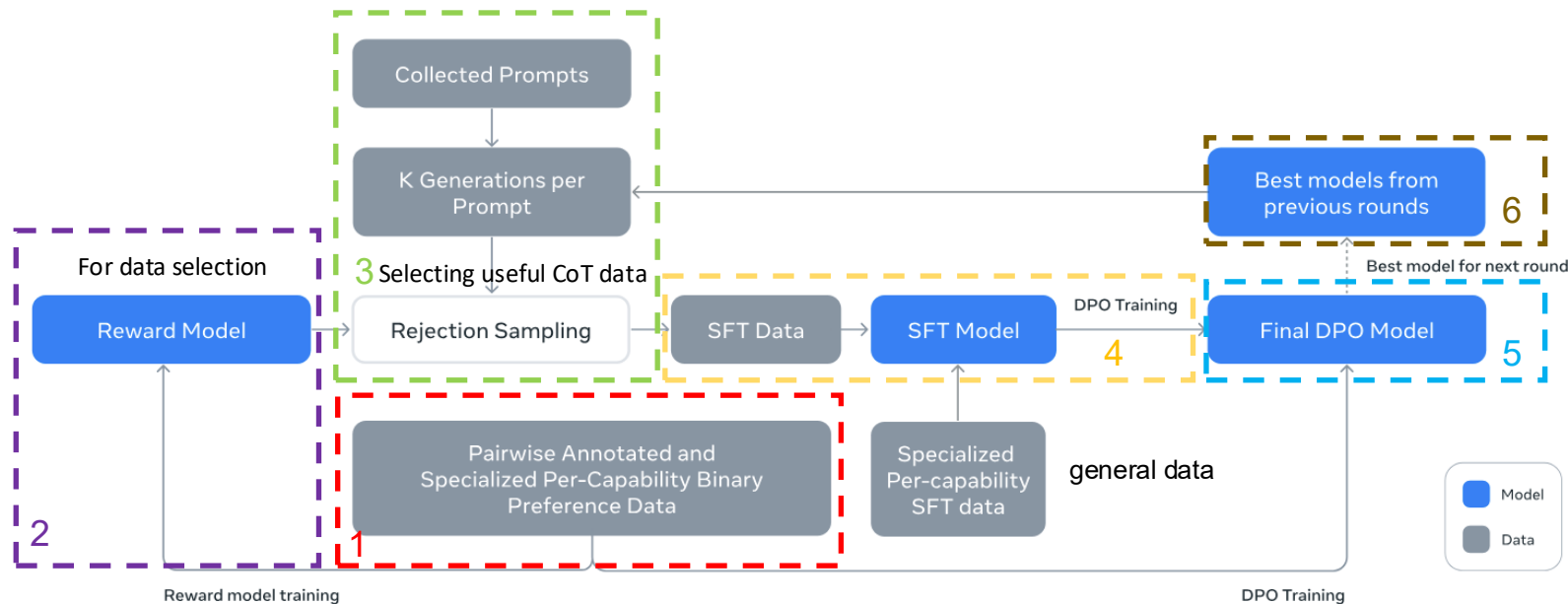


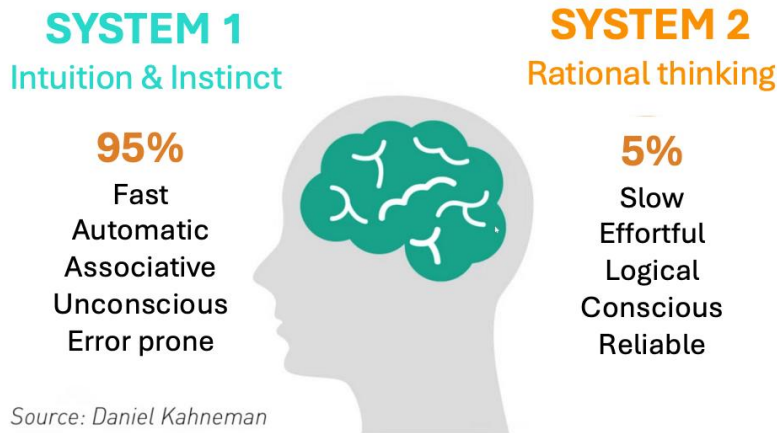
Illustration of the Overall Post-training Approach for Llama 3



Begin training the reward model using a small set of initial data (cold start data).

The Shift from Next-Token Prediction to Reasoning Models

- Traditional LLMs utilize next-token prediction aligned with fast, intuitive System 1 thinking.
- Reasoning models like OpenAI's o1 and DeepSeek R1 employ RL on Chain-of-Thought reasoning, embodying slow, deliberate System 2 thinking suited for tasks like research and planning.



Source: Daniel Kahneman

DeepSeek-R1-Zero Training Pipeline

- How RL - GRPO Works:
 - Sampling Outputs: For a given input problem, the model generates multiple outputs, each containing a reasoning process and a final answer.
 - Reward Calculation: GRPO (Group Relative Policy Optimization) evaluates each output using predefined rules, e.g., accuracy rewards and format rewards, to calculate rewards.

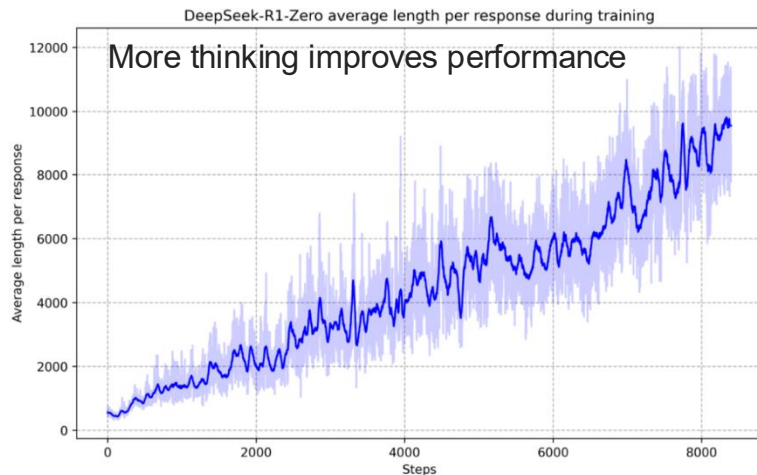
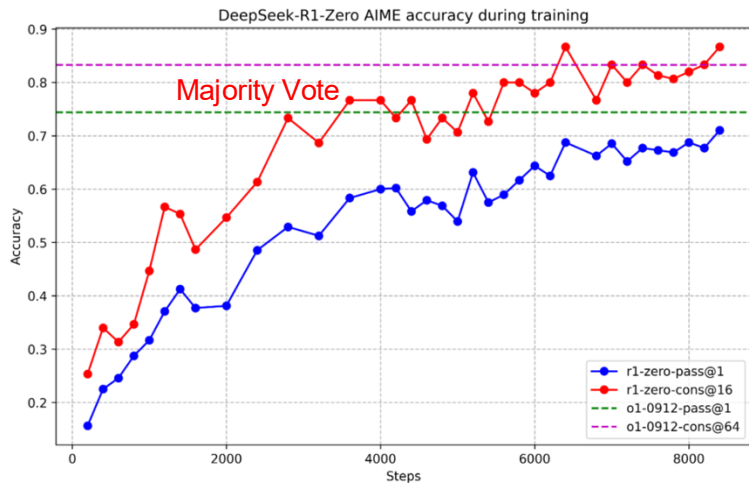


- Template used for DeepSeek-R1-Zero

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within `<think>` `</think>` and `<answer>` `</answer>` tags, respectively, i.e., `<think>` reasoning process here `</think>` `<answer>` answer here `</answer>`. User: **prompt**. Assistant:

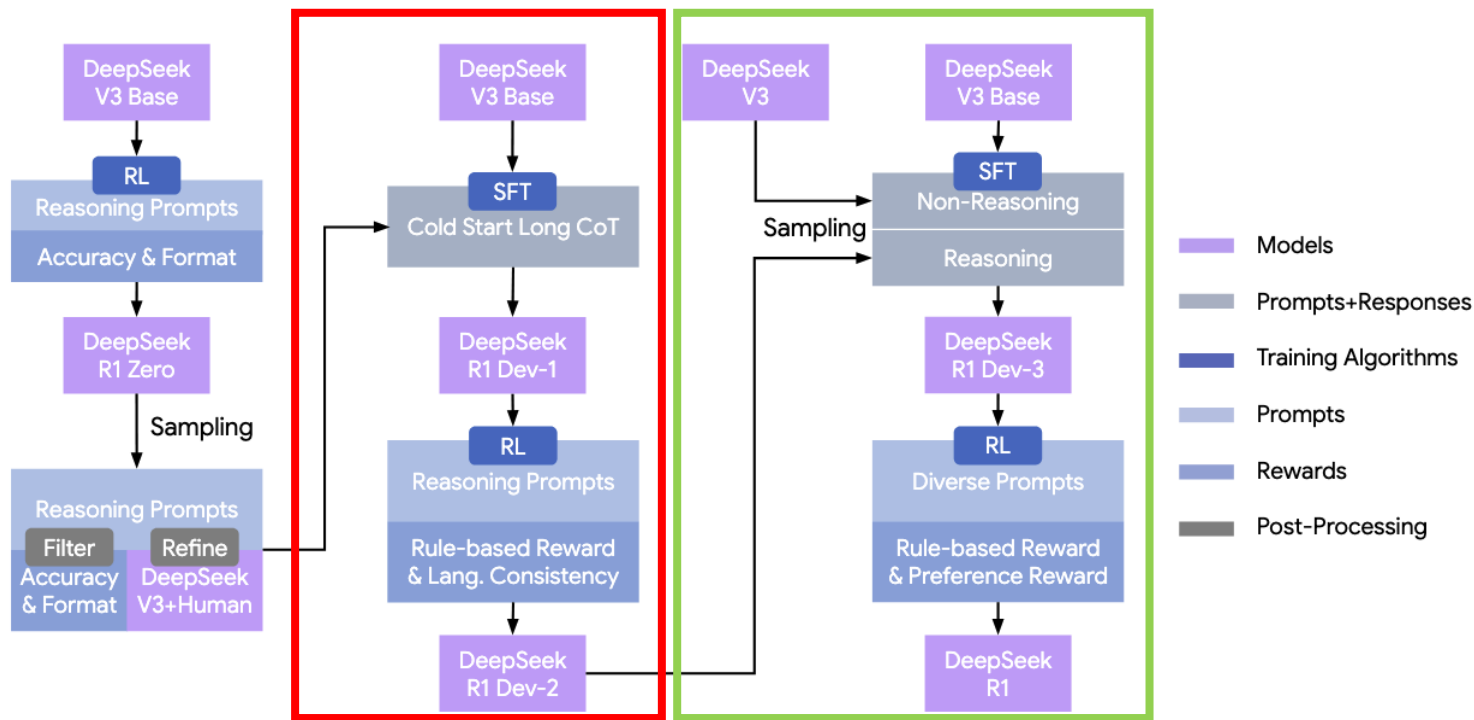
Key Findings:

- DeepSeek-R1-Zero demonstrates that advanced, human-like reasoning capabilities—such as, chain-of-thought (CoT), self-reflection, and self-correction—can emerge in LLMs purely through RL without any SFT.
- Self-Evolution: As training progresses, the model naturally learns to allocate more “thinking time” for reasoning tasks, evidenced by the increasing length of responses.
- Drawbacks: poor readability, language mixing, endless repetition, overthinking simple tasks, etc.



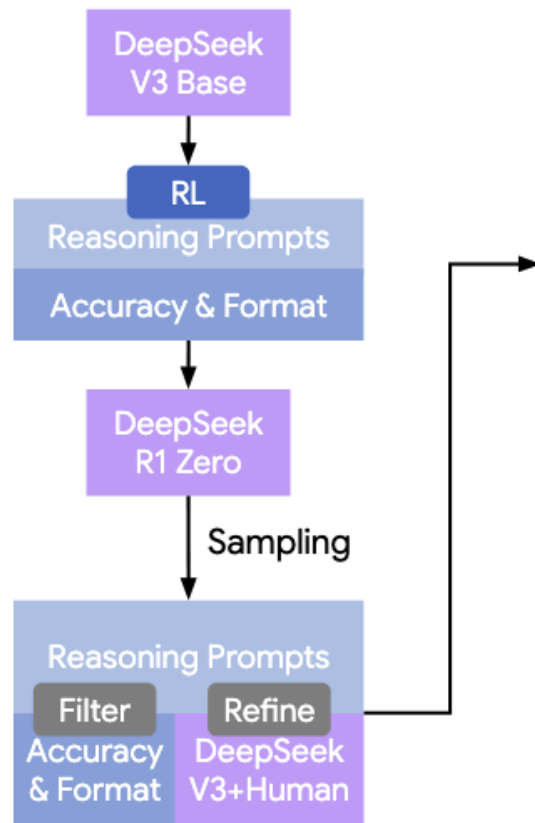
DeepSeek-R1 Training Pipeline

- First Round: Cold Start SFT
- Second Round: Rejection Sampling SFT and Auxiliary RL



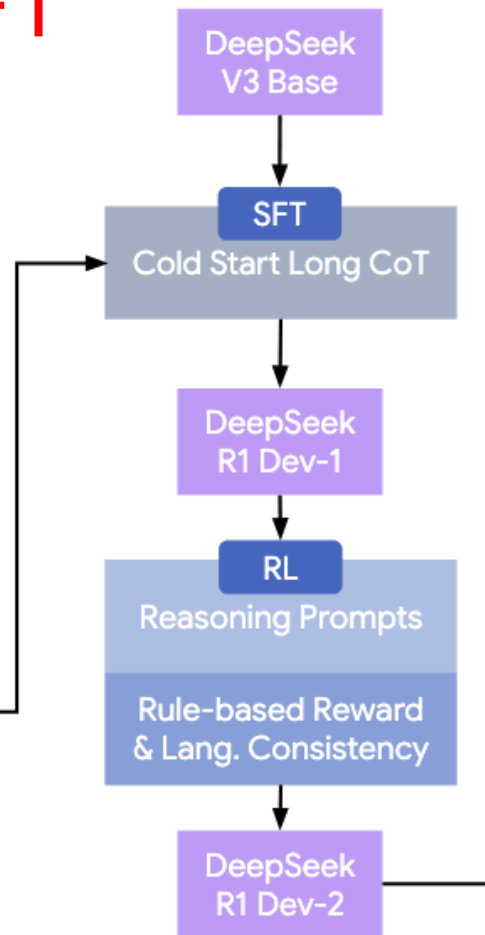
PoC - DeepSeek-R1-Zero

- This path represents the initial proof-of-concept.
- Process: They took the raw DeepSeek-V3 Base model and applied pure RL.
- Rewards: The model was rewarded solely for Accuracy (getting the right answer) and Format (following structure).
- Result: It achieved strong reasoning but suffered from "messy" outputs (e.g., mixing languages, poor readability), which motivated the main R1 pipeline.



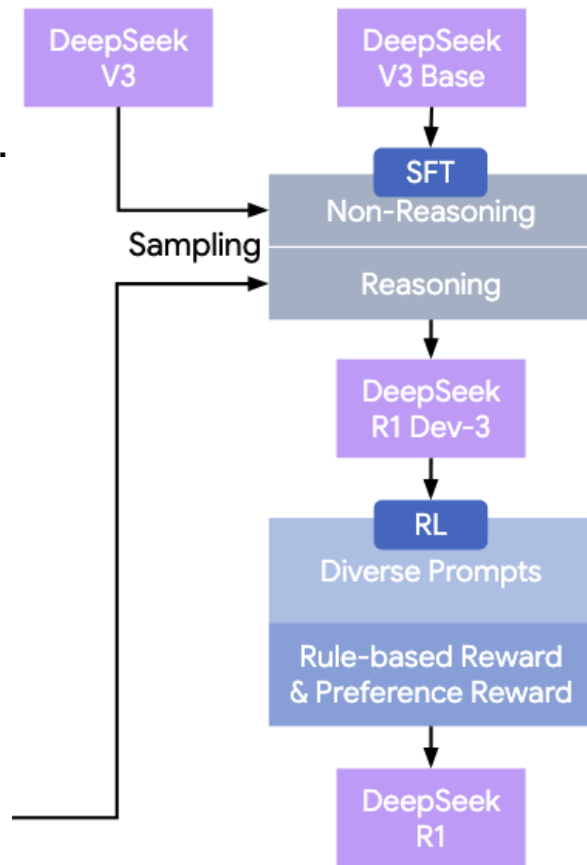
1st Round: Cold Start SFT

- Cold Start (SFT):
 - Input: DeepSeek-V3 Base.
 - Action: SFT using a small set of high-quality "Chain of Thought" (CoT) examples.
 - Goal: Teach the model *how* to present its thinking clearly before letting it explore.
 - Output: **DeepSeek-R1 Dev-1**.
- Reasoning RL:
 - Input: **DeepSeek-R1 Dev-1**.
 - Action: RL training with rewards for correctness + Language Consistency (to fix the language mixing)
 - Goal: Boost reasoning capabilities while maintaining readable text.
 - Output: **DeepSeek-R1 Dev-2**.

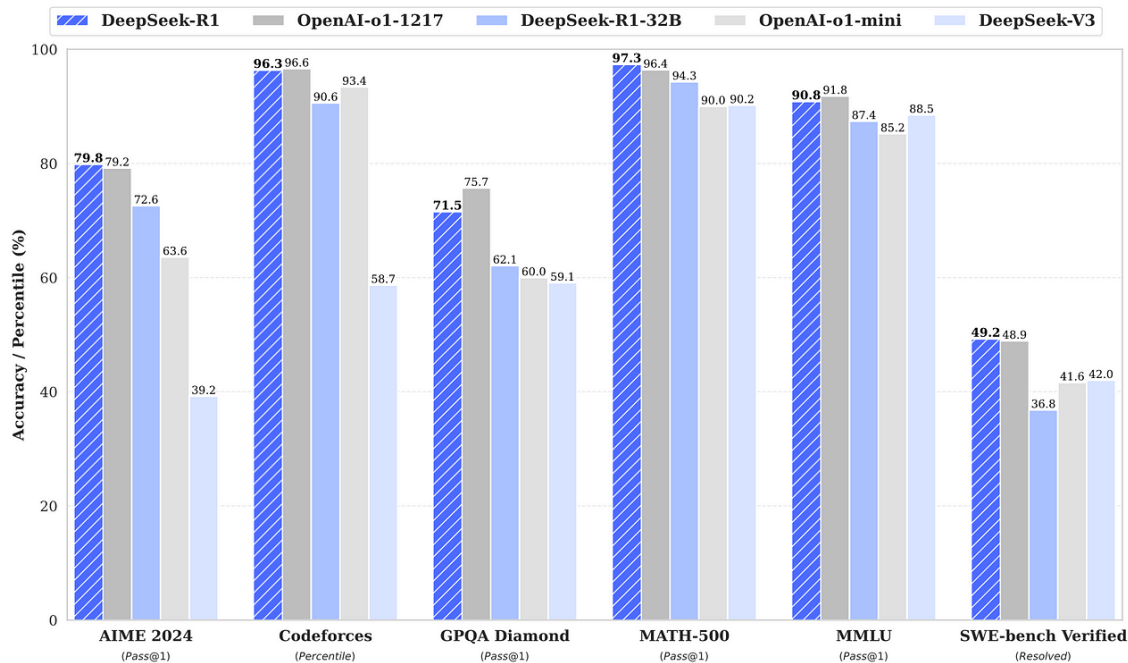


2nd Round: Rejection Sampling SFT & RL

- Rejection Sampling & SFT (The "Reset"):
 - Action: Dev-2 model was used to generate massive amounts of reasoning data , ~ **600K**. This data undergoes filtering through Rejection Sampling and is merged with ~**200K** of general non-reasoning data, which comes directly from DeepSeek-V3 SFT dataset.
 - Critical Step: They take a fresh DeepSeek-V3 Base model and fine-tune it on this combined dataset.
 - Goal: Imprint the reasoning patterns into a stable general-purpose model.
 - Output: DeepSeek-R1 Dev-3.
- Final Alignment RL:
 - Input: DeepSeek-R1 Dev-3.
 - Action: A final RL stage using multiple reward types (Rule-based for math/code + Preference Rewards for helpfulness/safety).
 - Goal: Polish the model for human interaction.
 - Output: The final DeepSeek-R1



Performance



- DeepSeek-R1 demonstrates competitive performance against state-of-the-art models like OpenAI's o1 across various benchmarks in 2025

Dataset	Primary Domain	Difficulty Level	Main Purpose
AIME 2024	Math	High School Competition	Multi-step reasoning
MATH-500	Math	General High School	Broad math proficiency
GPQA Diamond	Science	PhD / Graduate	Expert-level reasoning
LiveCodeBench	Coding	Software / Contests	Contamination-free coding
Codeforces	Coding	Competitive Programming	Algorithmic logic

Distilled Model Evaluation

Distilling more powerful models into smaller ones yields excellent results.

Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
GPT-4o-0513	9.3	13.4	74.6	49.9	32.9	759
Claude-3.5-Sonnet-1022	16.0	26.7	78.3	65.0	38.9	717
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	1820
QwQ-32B-Preview	50.0	60.0	90.6	54.5	41.9	1316
DeepSeek-R1-Distill-Qwen-1.5B	28.9	52.7	83.9	33.8	16.9	954
DeepSeek-R1-Distill-Qwen-7B	55.5	83.3	92.8	49.1	37.6	1189
DeepSeek-R1-Distill-Qwen-14B	69.7	80.0	93.9	59.1	53.1	1481
DeepSeek-R1-Distill-Qwen-32B	72.6	83.3	94.3	62.1	57.2	1691
DeepSeek-R1-Distill-Llama-8B	50.4	80.0	89.1	49.0	39.6	1205
DeepSeek-R1-Distill-Llama-70B	70.0	86.7	94.5	65.2	57.5	1633

Distillation to Smaller Models

- In 2022, Google research showed that the distilled 62 billion model exceeds the pre-trained 540 billion model, while the 8 billion model surpasses the pre-trained 62 billion model, indicating that larger models' reasoning knowledge can significantly enhance the performance of smaller models in resource-constrained

Language Model after Self-Improvement (LMSI)	Results on GSM8K		
	8 billion	62 billion	540 billion
w/o LMSI	5.0	29.7	56.5
Distilled from LMSI 540 billion	33.4	57.4	-

S1: Simple Test-time Scaling

- Data quality is more important than data quantity.

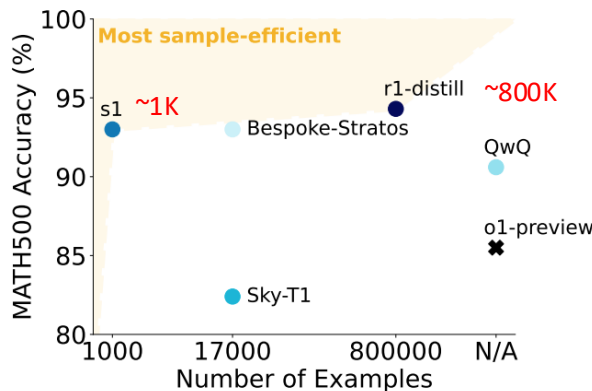
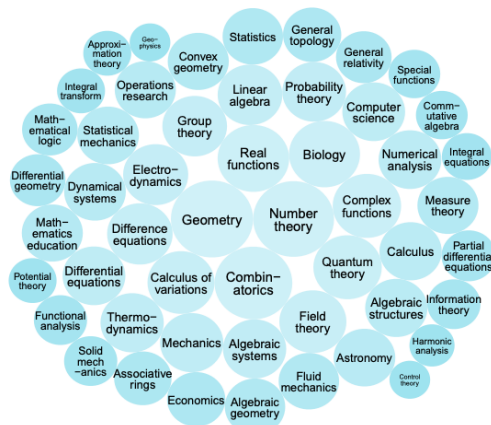


Figure 2. **s1K** and **s1-32B**. (left) **s1K** is a dataset of 1,000 high-quality, diverse, and difficult questions with reasoning traces. (right) **s1-32B**, a 32B parameter model finetuned on **s1K** is on the sample-efficiency frontier. See Table 1 for details on other models.

Budget Forcing for Long CoT

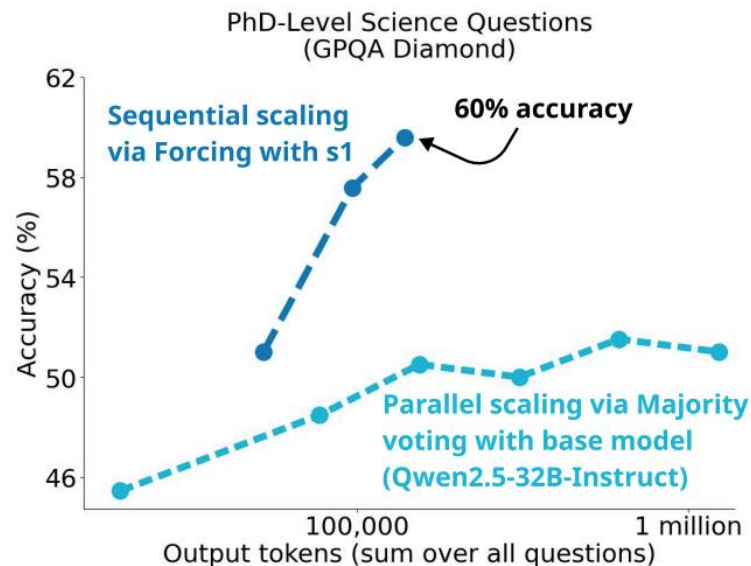
- Sequential scaling via budget forcing (Extending Thought with 'Wait' token) is proposed to generate long CoT.

How many r in raspberry? **Question**

Let's break down the process of counting the letter 'r' in the word "raspberry" ...
* First letter: 'r' - This is an 'r', count = 1.
* Second letter: 'a' - Not an 'r', count remains 1 ...
* Sixth letter: 'e' - Not an 'r', count remains 1.
* Seventh letter: 'r' - This is an 'r', count = 2.
* Eighth letter: 'y' - Not an 'r', count remains 2 ...
The number of 'r's in "raspberry" is 2.
Wait, let's re-read the question carefully. It asks "How many r in raspberry?" ... * r - a - s - p - b - e - r - r - y ... * First 'r' ... * Second 'r' ... * Third 'r' ... Count = 3 ... **Reasoning trace**

My initial answer of 2 was incorrect due to a quick reading of the word. **Final Answer:** The final answer is **3** **Response**

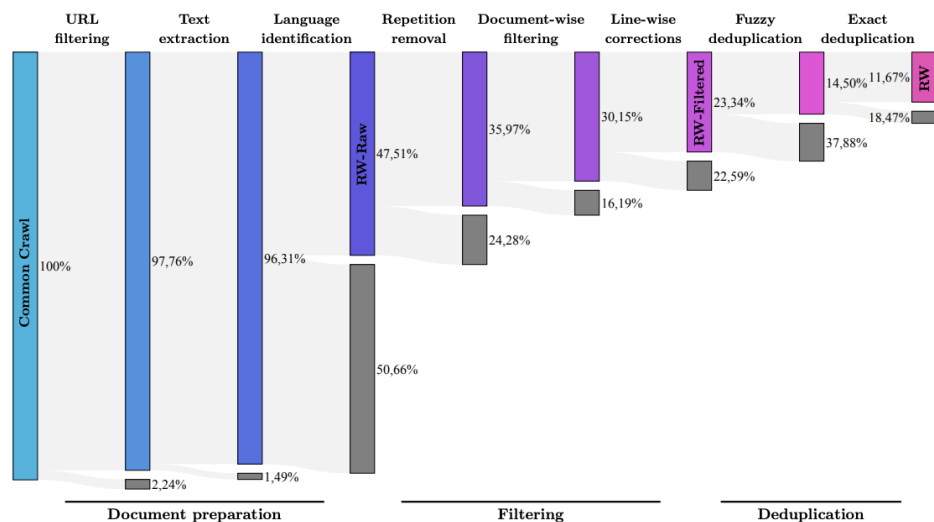
Figure 3. Budget forcing with s1-32B. The model tries to stop after "...is 2.", but we suppress the end-of-thinking token delimiter instead appending "Wait" leading **s1-32B** to self-correct its answer.



(b) Parallel scaling via majority voting

Macrodata Refinement Pipeline

- The Macrodata Refinement pipeline transforms raw web data from CommonCrawl into the high-quality RefinedWeb dataset.
- Strict processing standards eliminate about 90% of original documents, retaining only the highest quality data



1. Document Preparation filters out bad URLs and non-English text, which alone discards about 50% of the documents.
2. Filtering, focuses on quality by removing machine-generated spam and repetitive text, narrowing the pool down to roughly 24% of the original data.
3. Deduplication stage removes identical or near-identical text using fuzzy and exact matching. The result is that only about 12% of the initial documents make it into the final dataset.

Data Collection for Pre-training

4. Retraining and Repeating: The uncollected pages found via these specific URL paths were added back into the Seed Corpus. The fastText model was then retrained on this larger, more diverse seed, and the process ran again. The authors repeated this cycle four times, ultimately collecting 120 billion math-related tokens.

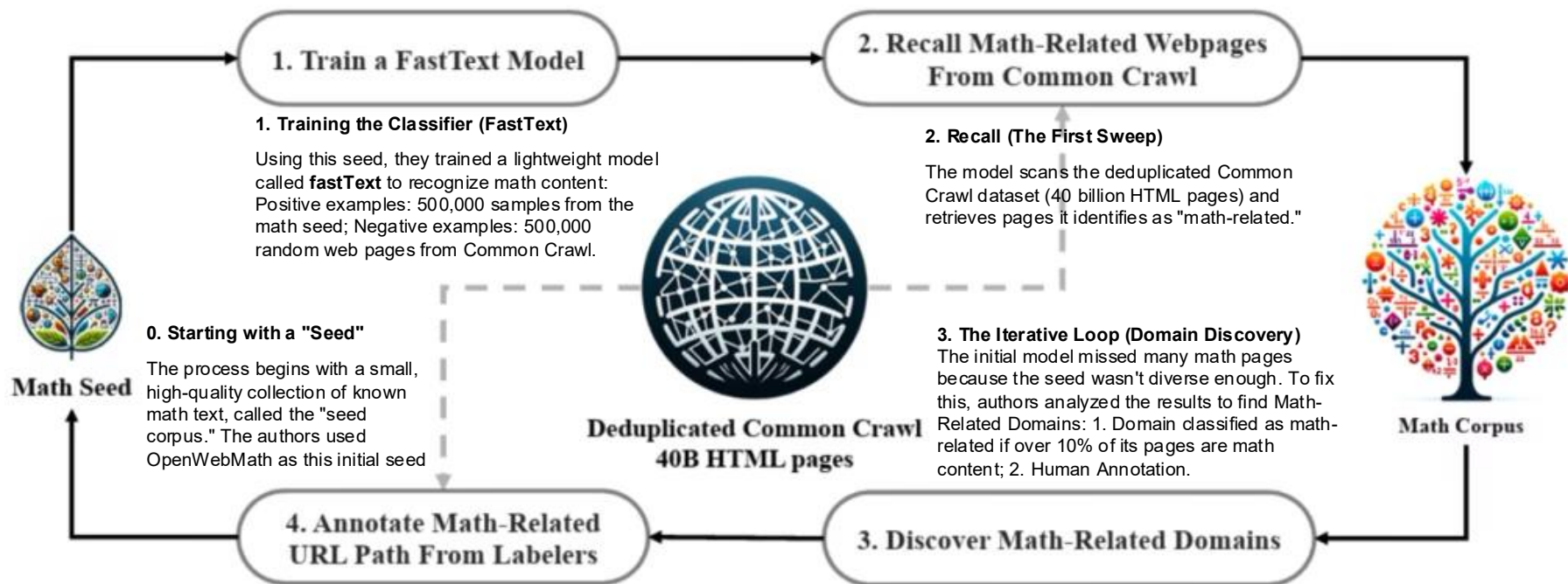


Figure 2 | An iterative pipeline that collects mathematical web pages from Common Crawl.

An Iterative Pipeline to Collect Math Web Pages from Common Crawl

- Round 1: A fastText classifier is trained using OpenWebMath as positive examples and random Common Crawl pages as negatives. The model scans the web to find top math-related pages.
- Round 2: Researchers analyze the domains of retrieved pages to identify math-focused websites (e.g., mathoverflow.net) and uncover specific URL paths with high-quality math content.
- Round 3: The newly identified high-quality pages are added to the training set, and the fastText model is retrained to improve its recognition of diverse mathematical content.
- Round 4: The refined model scans the web again, achieving 98% overlap with the previous round's results, indicating convergence, prompting the researchers to stop the process.

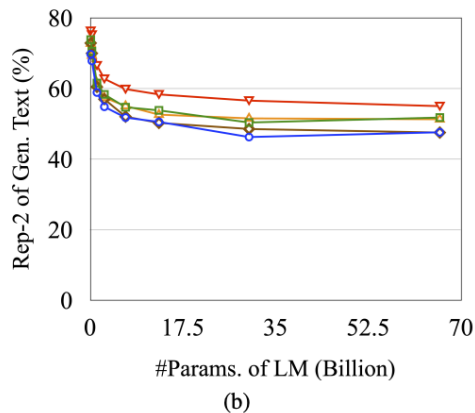
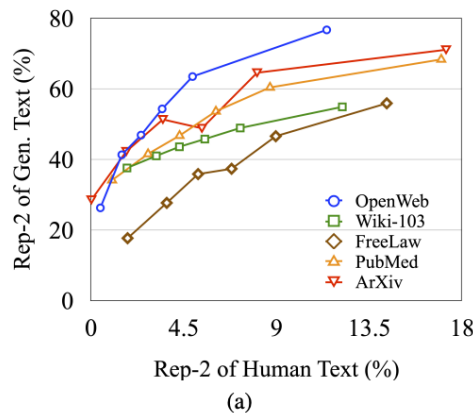
Repetition In Repetition Out

🧠: It was not possible to know which grade of general an officer was by his collar insignia . However , major generals and lieutenant **generals were two rows of nine buttons in groups of three** down the of the overcoat , and brigadier **generals were two rows of eight buttons in groups of two** .

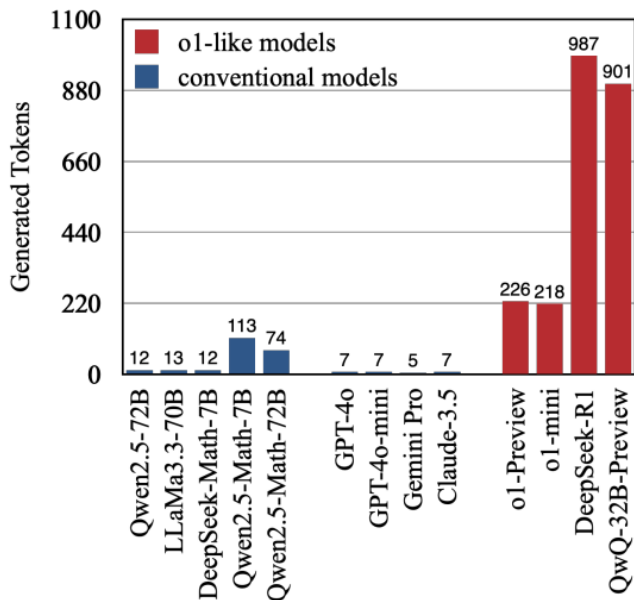
🤖: It was not possible to know which grade of general an officer would be required to be given to a particular officer . **The Army was not aware of the Army's role in the conflict , and the Army was not aware of the Army's role in the conflict . The Army's role in the conflict was limited to the training of**

The **blue** words and **red** words indicate the repetitions in human text and machine-generated text, respectively.

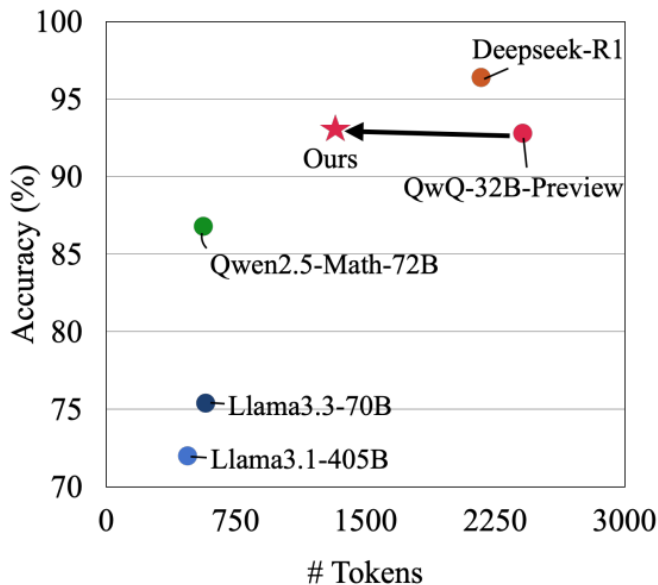
- LMs do not just mirror the repetition in training data but can actually amplify it by over 10 times
- Moreover, making models larger has diminishing returns, as the reduction in repetition eventually stalls even with massive scaling.



On the Overthinking of o1-Like LLMs



(a) Generated tokens on question "what is the answer of 2 plus 3?"



(b) Token-accuracy plot on MATH500

Figure 1: Illustration of **overthinking issue** in Figure (a): o1-like models (right panel) spend much more tokens than conventional LLMs (left and middle panels). Our method reduces the overthinking issue when applied to QwQ-32B-Preview (Figure (b)).

On the Underthinking of o1-Like LLMs

- A simple solution is to penalize the generated token “Alternatively”

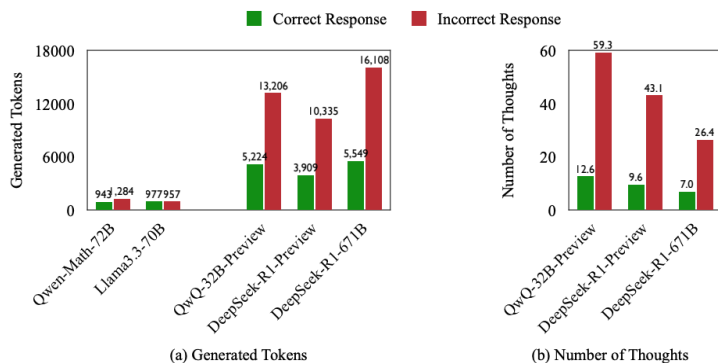


Figure 1: Illustration of the **underthinking issue** on the challenging AIME2024 testset: In o1-like models (e.g., QwQ-32B-Preview and DeepSeek-R1-671B), incorrect answers often switch reasoning strategies more frequently than correct ones (Figure b), leading to longer responses without improved accuracy (Figure a). In contrast, conventional LLMs (e.g., Qwen-Math-72B and Llama3.3-70B) show no significant difference in response length between incorrect and correct answers.

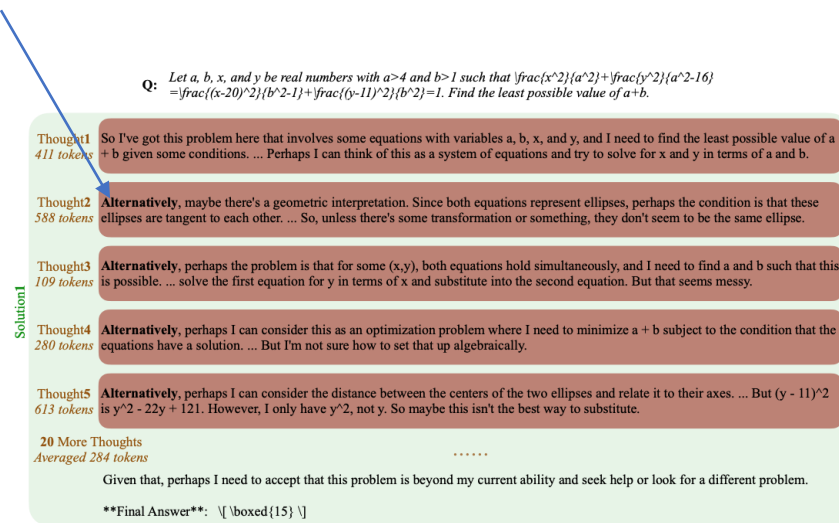


Figure 2: An example of underthinking issue for QwQ-32B-Preview model's output response that consists of 25 reasoning thoughts within a single solution.